

THE LAYER EVERY RETRIEVAL SYSTEM STANDS ON

Embeddings, From Text to Vectors

*How they are made, how they are compared,
what they quietly cannot do, and how to pick
one.*

LEARN EVERYTHING VISUALLY

What an Embedding Is

THE IDEA



this is 8 of 1536 numbers

one sentence becomes one point in space



Text becomes one point in space

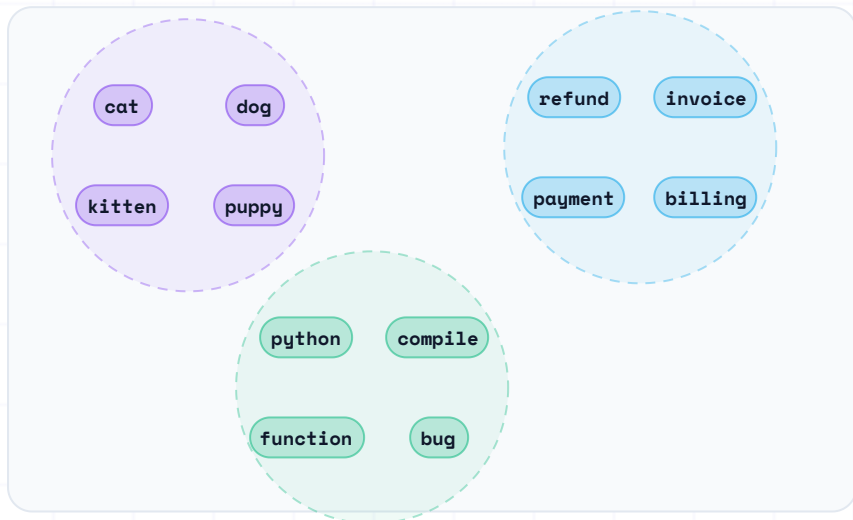


And distance becomes meaning

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

The Space

INTUITION



nobody placed these, the model learned the layout



Similar meanings land close together



Nobody labelled these regions



Real space has 1536 axes, not 2



Search is just finding neighbours

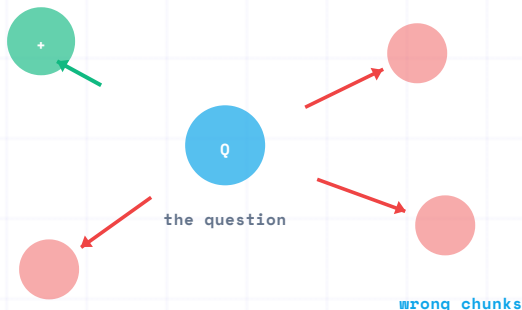
↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

How It Learns

CONTRASTIVE

PULL WHAT MATCHES CLOSER, PUSH THE REST AWAY

the right chunk



millions of these pairs, and the space arranges itself

+ Pull the matching pair together

— Push everything else apart

☰ Trained on millions of question and passage pairs

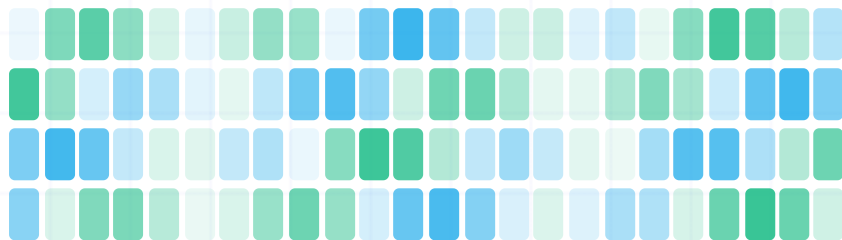
👁 Nobody defines the axes, they emerge

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Dimensions

SHAPE

ONE VECTOR, DRAWN AS A STRIP



96 of 1536 dimensions

1536

numbers

6 KB

per vector

0

you can name

no single dimension means anything on its own



No dimension means
anything alone



Meaning lives in the whole
pattern



More dimensions is more
memory



And the count is fixed by the
model

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Measuring Closeness

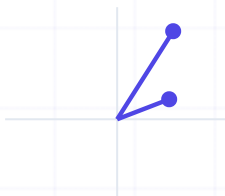
METRICS

COSINE



angle only

DOT



angle and length

L2



straight line gap

normalise the vectors and cosine and dot are identical

so pick one and use it everywhere

the metric must match the one used at index time

✓ Normalise once at write time



Then cosine and dot are the same thing



Mixing metrics breaks recall silently



Use whichever the model card recommends

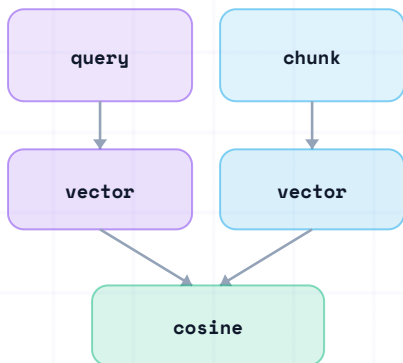
Normalised vectors turn every search into one matrix multiply.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Bi Encoder or Cross Encoder

TWO MODELS

BI ENCODER

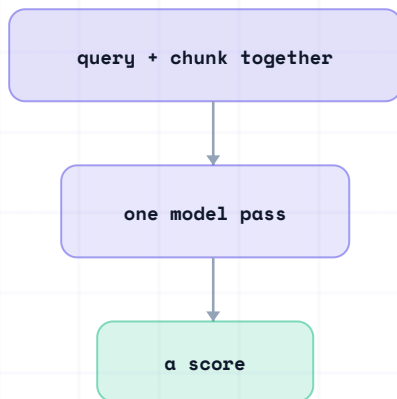


chunks embedded once

search is a dot product

fast, less accurate

CROSS ENCODER



one pass per pair

slow, more accurate

retrieve with the first, rerank with the second



One is precomputed, one is not



So you use both, in that order

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Symmetric or Asymmetric

SHAPE MISMATCH

SYMMETRIC

sentence

sentence

both sides look alike

dedupe, clustering

ASYMMETRIC

short question

a long passage of text that answers it

shapes do not match

this is what RAG is

a model trained for one is worse at the other

check which one your model was built for



A short question, a long passage



The wrong model quietly costs recall



RAG is always asymmetric



Dedupe and clustering are symmetric

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Prefix

INSTRUCTIONS

SOME MODELS WANT TO BE TOLD WHICH SIDE IT IS

```
embed("what is the refund window")  
  
embed("refunds are accepted for 14 days")
```



```
embed("query: what is the refund window")  
  
embed("passage: refunds are accepted for 14 days")
```



the prefix is not decoration, it changes the vector

and getting it wrong silently lowers recall



Check the model card
before you index



Nothing errors if you get it
wrong



Getting it wrong means a full
reindex



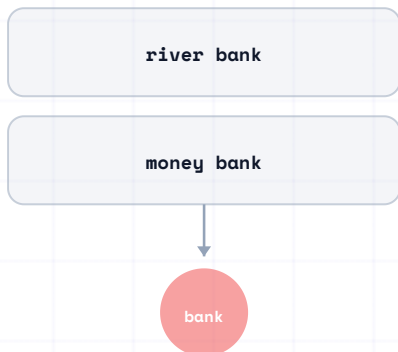
Test it with one known
question first

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Static or Contextual

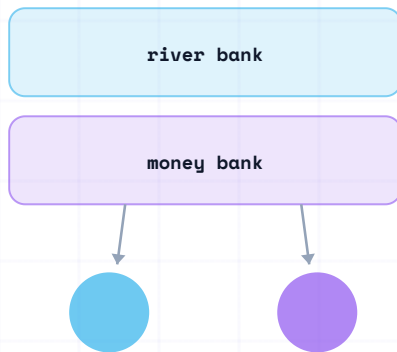
MEANING

STATIC



one vector for both

CONTEXTUAL



two different vectors

everything modern is contextual, and this is why



The sentence changes the vector



word2vec could not do this



Static was instant, this needs a pass



Worth it, and nobody argues otherwise

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Chunk Size Decides Quality

CHUNKING

TOO SMALL



no context



RIGHT



one idea



TOO BIG



diluted



one vector has to represent the whole chunk
so a chunk with three topics has a vector
that matches none of them well

800 characters with 120 overlap is the usual start



One vector must carry the whole chunk



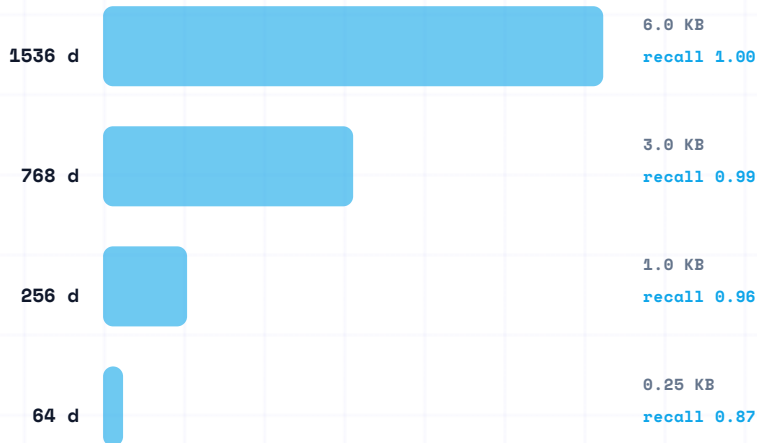
So chunking is an embedding decision

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Cutting the Vector Short

MATRYOSHKA

SOME MODELS LET YOU JUST CUT THE VECTOR SHORT



a quarter of the memory for a few percent of recall

only works on models trained for it



Truncate the tail, keep the head



Only on models trained this way



Store full, search truncated, rescore full



Measure the recall you actually lose

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Shrinking Each Number

QUANTISATION

SHRINK EACH NUMBER, KEEP MOST OF THE RECALL

float32



recall 1.00

float16



recall 1.00

int8



recall 0.98

binary



recall 0.90

int8 is almost free: 4x smaller, 2 percent of recall

binary needs a rescore pass to be usable

memory is what a vector database actually bills you for



int8 is the free win here



Binary needs a rescore pass

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

How Much Memory

SIZING

THE ONLY SIZING FORMULA YOU NEED

$$\text{vectors} \times \text{dimensions} \times \text{bytes} \times 1.5$$

100k chunks, 1536 d, float32

0.9 GB

1M chunks, 1536 d, float32

9.2 GB

10M chunks, 1536 d, float32

92 GB

10M chunks, 768 d, int8

11 GB

the 1.5 is the index overhead everyone forgets



Run this before choosing a store



Under 50M vectors, pgvector is usually enough

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

What They Cannot Do

BLIND SPOTS

THESE PAIRS LOOK ALMOST IDENTICAL TO AN EMBEDDING

not a refund

vs

a refund



negation is nearly invisible

order 88213

vs

order 88214



digits carry almost no signal

SKU-A9

vs

SKU-B7



exact ids blur together

this is the entire argument for hybrid search

BM25 catches exactly what embeddings blur



Negation, numbers and
exact ids



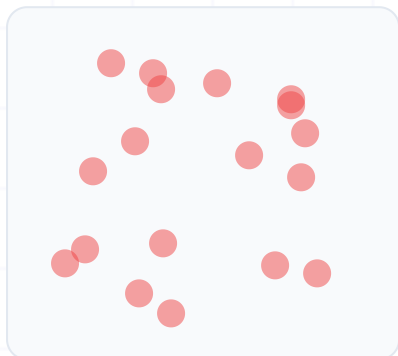
Which is why you add
keyword search

[↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT](#)

Domain Mismatch

JARGON

GENERAL MODEL



your jargon all lands
in one blurry cloud

FINE TUNED



the same terms separate
into real groups

fine tune when your domain vocabulary is not English



Test on your own vocabulary,
not a benchmark



Fine tuning is a last resort,
but a real one



A few thousand labelled
pairs is often enough



Try hybrid search before you
try training

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

What Actually Decides It

SELECTION

what language

multilingual or English only

symmetric or asymmetric

match it to the job

dimensions

memory versus recall

context length

how big can a chunk be

licence and hosting

API or run it yourself

MTEB score

a starting point, not an answer

benchmark on your own 50 questions, then decide



A leaderboard rank is a
shortlist, not a decision



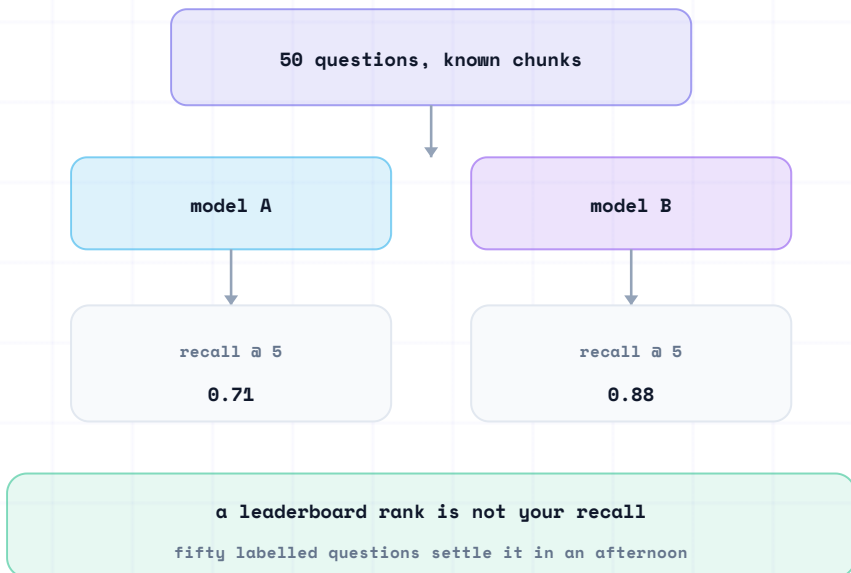
And the choice is expensive
to reverse

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Proving It

EVALUATION

JUDGE AN EMBEDDING MODEL ON YOUR DATA



Fifty labelled questions is enough



Compare recall at 5, not vibes



An afternoon of labelling, reused forever



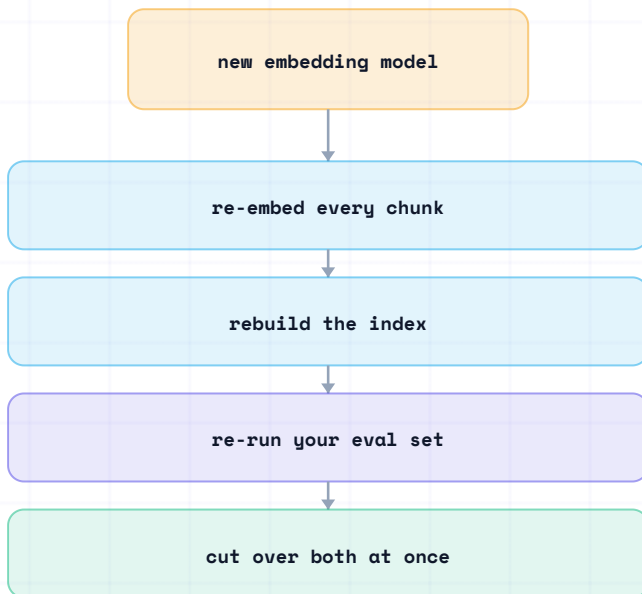
And it settles arguments that cost weeks

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Changing the Model

REINDEX

CHANGING THE MODEL IS NOT A CONFIG CHANGE



a query embedded with model B cannot search
an index built with model A

⚠ Never mix two models in one index

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Decisions in Order

SEQUENCE

STEP	DECIDE
1	Symmetric or asymmetric, and the language
2	Chunk size and overlap
3	The model, benchmarked on your questions
4	Dimensions and quantisation, from the memory math
5	Add keyword search alongside it
6	Add a reranker on top

↓ Each one constrains the next

↺ Step 3 is the expensive one to redo

📏 Measure after every step

👉 And stop as soon as recall is good enough

Steps 5 and 6 usually move recall more than any embedding model change does.

Most teams spend their time on step 3 and skip 5 and 6 entirely.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

Where Teams Go Wrong

MISTAKES

MISTAKE	WHAT HAPPENS
Different model at index and query	Recall collapses, nothing errors
Missing the query prefix	Quietly worse results
Chunks with three topics in them	A vector that matches none of them
Vectors only, no keyword search	Ids and codes are never found
Choosing by leaderboard alone	A model tuned for someone else data
No plan for reindexing	Stuck on the first model you picked

THE TWO THAT FAIL SILENTLY

⚠ A mismatched model

🔍 A missing prefix

Neither one throws an error. Recall just drops, and it looks like the model is bad.

Both are caught by one test: embed a known question, check the top hit.

↓ CHECK THE CAPTION FOR THE FULL INTERVIEW KIT

The Embedding Toolkit



Text to vectors



Contrastive training



Cosine and dot



Bi vs cross encoder



Asymmetric search



Query prefixes



Chunk size



Matryoshka



Quantisation



Memory math



Hybrid search



Reindex planning

Anyone can call an embedding API.
Engineers know what the vector
can and cannot tell apart.

Save this before your next retrieval build.